



REQUERIMIENTO FUNCIONAL RF13

Contratos inteligentes mediante chaincode

Informe de implementación, integración y validación

ESTADO	CHAINCODE	FECHA
Completado	v2.2 / secuencia 4	9 de agosto de 2026

PostgreSQL conserva la operación del sistema y Hyperledger Fabric protege la evidencia criptográfica y las decisiones automáticas de trazabilidad.

Trazaap | Trazabilidad alimentaria segura

Resumen ejecutivo

Se implementó el requerimiento RF13 para que Hyperledger Fabric no se limite a almacenar hashes, sino que ejecute reglas de negocio mediante el chaincode traceability. La solución protege la inmutabilidad de los eventos, bloquea despachos no conformes, registra decisiones auditables, permite confirmar la recepción del cliente y genera alertas de vencimiento.

La integración conserva la arquitectura híbrida del proyecto: PostgreSQL sigue siendo la base operativa, mientras que Fabric actúa como capa de integridad, auditoría y confianza. No se creó una tabla `bloque_blockchain` ni una simulación del ledger en la base relacional.

Resultado principal: Una liberación aprobada solo se convierte en despacho cuando el chaincode autoriza todas las reglas. Si Fabric no está disponible, el sistema responde HTTP 503 y no modifica el inventario.

Contenido del informe

- Arquitectura y decisiones de diseño.
- Funciones implementadas en el chaincode.
- Integración con backend, frontend y trazabilidad.
- Flujos automáticos de RF13.
- Pruebas, despliegue y evidencia de cumplimiento.
- Guía de demostración y recomendaciones siguientes.

1. Arquitectura implementada

La solución separa claramente los datos operativos de la evidencia inmutable. Esta decisión evita usar Fabric como base de consultas administrativas y evita que PostgreSQL intente comportarse como una blockchain.

Componente	Responsabilidad
PostgreSQL	Usuarios, proveedores, materias primas, órdenes, manufactura, liberación e inventarios.
Backend Node.js/Express	Normaliza payloads, aplica reglas operativas y coordina PostgreSQL con Fabric.
Chaincode traceability	Valida reglas, calcula hashes, registra evidencia y decisiones inmutables.
Hyperledger Fabric	Ledger permissionado con dos peers, orderer, CA, canal e identidad MSP.
Frontend Next.js	Muestra bloqueos, estados blockchain, alertas, confirmaciones y reportes.

1.1 Flujo de comunicación

1. El usuario registra una operación desde el frontend.
2. El backend valida los datos y guarda la información operativa en PostgreSQL.
3. El backend construye un payload estable con los datos funcionales del evento.
4. El chaincode canoniza el payload, calcula SHA-256 y registra la evidencia en Fabric.

5. La trazabilidad reconstruye el payload actual y solicita al chaincode verificar su integridad.
6. El frontend muestra VERIFICADO, VERIFICADO_CORREGIDO, ALTERADO o NO_ENCONTRADO.

Regla de separación: PostgreSQL = operación completa. Hyperledger Fabric = evidencia criptográfica, decisiones y auditoría.

2. Chaincode implementado

El contrato se encuentra en `fabric/chaincode/traceability/index.js`. La versión desplegada es 2.2 y la definición comprometida en el canal utiliza la secuencia 4.

Función	Qué hace
registrarEvento	Registra un evento nuevo y rechaza cualquier intento de sobrescritura.
validarEvento	Compara el payload actual con el original o con la última corrección autorizada.
registrarCorreccionEvento	Crea una evidencia nueva vinculada al evento original sin sustituirlo.
consultarHistorialEvento	Retorna el original y su secuencia de correcciones inmutables.
validarDespacho	Evalúa vencimiento, inventario, manufactura, liberación, transporte y controles críticos.
registrarDespacho	Registra el despacho únicamente cuando todas las reglas son conformes.
confirmarRecepcionCliente	Registra una recepción única después de validar lote y credenciales externas.
registrarAlertaVencimiento	Registra una sola alerta por lote vencido con unidades disponibles.
consultarEvento / consultarEventosPorLote	Permite recuperar evidencia individual o reconstruir el historial del lote.

2.1 Inmutabilidad y correcciones

La clave principal de un evento mantiene la forma `tipoEvento:idEntidad`. Si esa clave ya existe, `registrarEvento` retorna `EVENTO_DUPLICADO`. Por tanto, ningún flujo normal puede reemplazar el hash ni el payload originalmente comprometido.

Cuando una modificación es legítima, el gerente registra una corrección con motivo, actor, payload corregido, hash, `txId`, timestamp e identidad Fabric. `validarEvento` compara el estado operativo con la corrección más reciente y retorna `VERIFICADO_CORREGIDO` si coincide.

Diferencia importante: Una modificación directa en PostgreSQL aparece como ALTERADO. Solo una corrección explícita y auditada puede establecer una nueva versión válida.

2.2 Identidad y metadatos Fabric

El chaincode no confía únicamente en el nombre de actor enviado por el backend. Cada registro incluye también `txId`, timestamp de la transacción, MSP del invocador e identidad Fabric obtenida desde el contexto de la red.

3. Integración del backend

La comunicación con Fabric se mantuvo desacoplada en el módulo backend/src/modulos/blockchain. fabric.client.js traduce los errores del Gateway y fabric-traceability.service.js ejecuta las transacciones y consultas del contrato. blockchain.service.js construye eventos, valida su estado y administra las correcciones.

3.1 Payloads estables

Cada tipo de evento tiene un constructor específico dentro de backend/src/modulos/blockchain/payloads. Estos constructores consultan las relaciones necesarias, excluyen metadatos técnicos y ordenan las claves para que la misma información produzca siempre el mismo hash.

Categoría	Tratamiento
Sí entran	Lotes, productos, proveedores, responsables, cantidades, tiempos, temperaturas, decisiones y observaciones funcionales.
No entran	created_at, updated_at, tokens, estados de interfaz, logs e identificadores técnicos dentro del payload.
Se usa aparte	idEntidad se utiliza para localizar el evento en Fabric, sin contaminar el contenido funcional.

3.2 Errores controlados

- Fabric no disponible: HTTP 503 en operaciones críticas.
- Evento duplicado: se informa sin sobrescribir la evidencia.
- Lote no encontrado: error funcional controlado.
- Despacho bloqueado: retorna todos los motivos y reglas evaluadas.
- Recepción ya confirmada y alerta duplicada: se rechazan de forma explícita.

4. Validación y registro del despacho

En Trazaap, la liberación aprobada representa actualmente el despacho. Antes de escribir una liberación aprobada, el backend construye los datos de validación y llama a validarDespacho. El chaincode recalcula cada regla y emite APROBADO o BLOQUEADO.

Grupo de reglas	Validación realizada
Manufactura	Existe manufactura y el lote coincide con el producto fabricado.
Estado	La liberación está aprobada y el lote no fue despachado antes.
Vencimiento	La fecha de vencimiento es posterior al momento de despacho.
Liberación	Etiqueta, lote, fecha, empaque, envase y estado del producto cumplen.
Transporte	Conductor, placa, limpieza del vehículo y documentación cumplen.
Producción	Fermentación, horneado e inmersión están dentro de los rangos calculados.
Inventario	Las unidades disponibles son suficientes para la cantidad despachada.

4.1 Transacción operativa

1. El backend solicita la decisión del chaincode antes de escribir en PostgreSQL.
2. Si la decisión es BLOQUEADO, conserva el formulario y muestra todos los motivos.
3. Si la decisión es APROBADO, inicia una transacción PostgreSQL.
4. Registra la liberación y el despacho en Fabric.
5. Crea o actualiza el inventario de producto terminado.
6. Confirma la transacción únicamente si Fabric respondió correctamente.

Si registrarDespacho falla, la transacción relacional se revierte. Esto evita liberaciones parciales, inventarios descontados sin evidencia o lotes marcados como entregados sin autorización.

4.2 Controles críticos

Las tolerancias se calculan en controles-criticos.service.js a partir de los valores estándar del producto y los valores reales de manufactura. El frontend no define reglas propias. El chaincode vuelve a verificar numéricamente cada mínimo, máximo y valor recibido.

Proceso	Tolerancia inicial
Fermentación	Tiempo: ± 10 %, mínimo 3 min. Temperatura: ± 3 °C.
Horneado	Tiempo: ± 10 %, mínimo 2 min. Temperatura: ± 8 °C.
Inmersión	Solo para Bagel. Tiempo: ± 15 %, mínimo 1 min. Temperatura: ± 5 °C.

5. Confirmación del cliente

El cliente receptor no necesita una cuenta interna. Desde el portal público del QR puede consultar el lote y confirmar la recepción mediante el número de factura o el código de cliente.

1. El backend valida las credenciales contra la información operativa del lote.
2. El chaincode comprueba que el lote existe y posee un despacho aprobado.
3. Se valida que la factura o el código correspondan al despacho.
4. Se rechaza cualquier confirmación duplicada.
5. Fabric registra RECIBIDO_POR_CLIENTE con fecha, actor y txId.

Privacidad: El portal del consumidor no expone hashes, identidades Fabric ni información comercial sensible. El detalle técnico permanece reservado para usuarios autorizados y el acceso de auditoría.

6. Alertas de vencimiento

Un chaincode no puede despertarse por sí mismo cuando pasa el tiempo. Por eso se agregó una tarea programada en vencimientos.job.js. El backend revisa periódicamente los lotes aprobados que ya vencieron, conservan unidades y no tienen despacho confirmado.

Cuando encuentra un caso, invoca registrarAlertaVencimiento. El chaincode evita duplicados y registra lote, producto, fecha de vencimiento, unidades disponibles, fecha de detección y estado VENCIDO_SIN_DESPACHO. El dashboard gerencial presenta las alertas activas.

7. Cambios en frontend, trazabilidad y reportes

Vista	Comportamiento agregado
Liberación	Muestra los motivos del chaincode dentro del formulario y conserva los datos para corregirlos.
Portal QR	Permite consulta pública y confirmación controlada por factura o código.
Dashboard	Presenta alertas de lotes vencidos sin despacho.
Trazabilidad interna	Muestra estado, hashes, mensaje, correcciones y decisiones automáticas.
Reporte PDF	Incluye validación, decisión, motivos, fecha Fabric y txId por etapa.

No se creó una pantalla separada para usar blockchain. La integración ocurre automáticamente dentro de los flujos operativos, tal como exige la arquitectura del proyecto.

8. Eventos cubiertos

Evento Fabric	Propósito
repcion_materia_prima	Registro inicial e integridad de la recepción.
inspeccion_recepcion	Resultado, condiciones y decisión de aceptación.
orden_produccion	Creación y versiones funcionales de la orden.
producto_fabricado_configurado	Ficha técnica, variante y receta vigente.
registro_manufactura	Lote producido, cantidades, tiempos y temperaturas reales.
liberacion_producto	Validaciones y decisión de liberación.
despacho_producto	Decisión automática y salida aprobada.
inventario_producto_terminado	Disponibilidad posterior a liberación.
inventario_materia_prima	Versiones funcionales del saldo de materias primas.
movimiento_inventario	Entradas, salidas y referencias operativas.
confirmacion_recepcion_cliente	Entrega confirmada por el receptor.
correccion_evento	Corrección vinculada al evento original.
alerta_vencimiento	Producto vencido con unidades y sin despacho.

9. Pruebas realizadas

La implementación se verificó con pruebas automatizadas, consultas reales a la red, compilación del frontend y revisión visual de las vistas.

Capa	Resultado	Cobertura
Chaincode	10 de 10	Inmutabilidad, alteración, correcciones, despacho, recepción y alertas.
Backend	13 de 13	HTTP 503, rollback, inventario, credenciales públicas, hashes y RBAC.
Frontend	17 rutas	Compilación optimizada sin errores de tipos ni renderizado.
Red Fabric	Correcto	Chaincode 2.2 / secuencia 4 confirmado desde peer0 y peer1.
Prueba visual	Correcto	Portal público, vista técnica, historial de correcciones y reporte.

9.1 Casos de chaincode

- Registrar un evento nuevo con identidad Fabric.
- Rechazar la sobrescritura de un evento existente.
- Detectar un registro operativo alterado.
- Registrar una corrección conservando el original.
- Confirmar una recepción y evitar la confirmación duplicada.
- Permitir un despacho conforme y bloquear controles fuera de rango.
- Bloquear un lote vencido y evitar alertas repetidas.

10. Despliegue de la red

Elemento	Configuración activa
Organización	Org1MSP
CA	ca.trazaap.local : 7054
Orderer	orderer.trazaap.local : 7050
Peer principal	peer0.org1.trazaap.local : 7051
Peer secundario	peer1.org1.trazaap.local : 8051
Canal	trazabilidad-channel
Chaincode	traceability 2.2, secuencia 4

El paquete se instaló en ambos peers, fue aprobado para Org1MSP y comprometido en el canal. El despliegue no eliminó el ledger, no regeneró certificados y no recreó el canal.

11. Matriz de aceptación de RF13

N.º	Criterio	Estado	Evidencia
1	Un evento existente no puede sobrescribirse.	Completado	EVENTO_DUPLICADO y prueba automatizada.
2	Una modificación aparece como ALTERADO.	Completado	Prueba unitaria y caso real de recepción 42.
3	Una corrección conserva el original.	Completado	Historial y VERIFICADO_CORREGIDO.
4	Un despacho fuera de rango se bloquea.	Completado	Reglas y motivos retornados por chaincode.
5	Un despacho conforme se registra en Fabric.	Completado	Prueba de registrarDespacho.
6	Inventario cambia solo tras aprobación.	Completado	Transacción y pruebas de no escritura.
7	Ciente confirma sin cuenta interna.	Completado	Portal público con factura o código.
8	Lote vencido genera una sola alerta.	Completado	Job periódico y deduplicación Fabric.
9	Trazabilidad y reporte muestran decisiones.	Completado	Vista técnica y reporte verificados.
10	Pruebas automatizadas pasan.	Completado	10/10 chaincode y 13/13 backend.
11	Red funcional con ambos peers.	Completado	Versión 2.2 / secuencia 4 en peer0 y peer1.
12	Frontend y backend continúan funcionando.	Completado	Build, health check y navegación real.

12. Evidencia real de la validación

Durante la verificación final se registró una corrección autorizada de la ficha técnica del producto con id 1. Fabric conservó el evento original, registró una nueva transacción y la trazabilidad pasó a VERIFICADO_CORREGIDO.

Dato	Resultado
Entidad	producto_fabricado_configurado:1
Estado	VERIFICADO_CORREGIDO
Motivo	Actualización autorizada de la ficha técnica y receta vigente.
Transacción	8258a46116a474514fd27fa8d13d126040064591e1f1c6759112b972f01ba4ef
Original	Conservado sin modificación.

Comportamiento correcto: La recepción histórica 42 permanece como ALTERADO porque su información ya no coincide con el ledger y no existe una corrección justificada. El sistema no oculta esa diferencia.

13. Cómo demostrar RF13

1. Iniciar Docker Desktop y comprobar que CA, orderer, peer0 y peer1 están activos.
2. Iniciar backend en el puerto 4000 y frontend en el puerto 3000.
3. Consultar un lote producido desde Trazabilidad.
4. Revisar la tabla Validación blockchain y el historial técnico de correcciones.
5. Abrir el reporte PDF y comprobar estados, decisiones, fecha Fabric y txId.
6. Intentar liberar un lote con un control fuera de rango y verificar el bloqueo.
7. Corregir los datos, aprobar el despacho y comprobar el registro Fabric.
8. Abrir el portal QR y confirmar la recepción con factura o código de cliente.

13.1 Comandos de verificación

```
cd fabric/chaincode/traceability
npm test
cd ../../../../backend && npm test
cd ../frontend && npm run build
```

Para actualizar el chaincode sobre un ledger existente se debe incrementar versión y secuencia. No se debe ejecutar clean.sh ni eliminar certificados para una actualización normal.

14. Archivos principales

Responsabilidad	Ubicación
Chaincode	fabric/chaincode/traceability/index.js
Pruebas chaincode	fabric/chaincode/traceability/test/traceability.test.js
Cliente Fabric	backend/src/modulos/blockchain/fabric.client.js
Servicio blockchain	backend/src/modulos/blockchain/blockchain.service.js
Liberación/despacho	backend/src/modulos/liberacion/liberacion.service.js
Controles críticos	backend/src/modulos/liberacion/controles-criticos.service.js
Alertas	backend/src/modulos/liberacion/vencimientos.job.js
Portal público	backend/src/modulos/publico y frontend/src/app/verificar/[lote]
Trazabilidad	backend/src/modulos/trazabilidad y frontend/src/modulos/trazabilidad
Despliegue	fabric/scripts/deploy-chaincode.sh y fabric/scripts/env.sh

15. Conclusión

RF13 quedó implementado como una capa real de contratos inteligentes sobre Hyperledger Fabric. El chaincode ya no es un almacenamiento pasivo de hashes: valida reglas, toma decisiones, protege la inmutabilidad y conserva evidencia auditable de cada resultado.

La solución mantiene la separación correcta entre la operación empresarial y la confianza criptográfica. Esto permite que Trazaap continúe usando PostgreSQL para su funcionamiento diario y utilice Fabric para demostrar integridad, trazabilidad y cumplimiento ante revisiones internas o procesos de auditoría.

Estado final: RF13 completado, probado y desplegado con dos peers, orderer, CA, canal y chaincode funcionales.